

Day 39–40: Date, Time API + Regular Expressions

Practice:

- Date formatting
- Pattern matching

PART 1 — Java Date & Time API

1. First: Date and Time ante enti?

Simple ga:

- **Date** → eeroju ye date?
- **Time** → ippudu entha time?
- **Date + Time** → exact moment

Example:

Date → 05-10-2026

Time → 06:30 PM

DateTime → 05-10-2026 06:30 PM

Java applications lo dates/times chala common.

Real-world examples

Bank application:

Transaction Date → 05-10-2026

Transaction Time → 06:30 PM

E-commerce:

Order Date → 05-10-2026

Expected Delivery → 08-10-2026

Student portal:

Exam Date → 15-10-2026

Login system:

Last Login → 05-10-2026 06:25 PM

So Java lo dates and times handle cheyyadaniki **Date & Time API** use chestham.

2. Old Java Date API vs New Date & Time API

Java lo historically Date and Calendar classes unnayi.

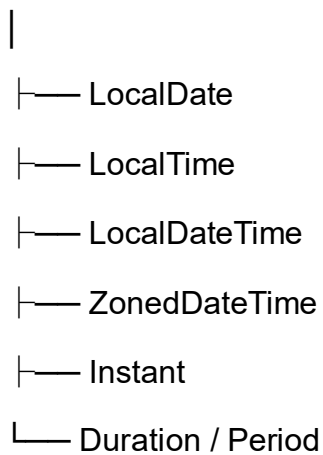
Old style:

Date date = new Date();

But modern Java lo **Java 8 nundi java.time package** introduce chesaru.

Main classes:

java.time



Day 39–40 ki especially important:

LocalDate

LocalTime

LocalDateTime

DateTimeFormatter

Period

Duration

3. LocalDate

LocalDate ante:

Only date, time lekunda.

Example:

2026-10-05

Java:

```
import java.time.LocalDate;

public class Main {

    public static void main(String[] args) {

        LocalDate today = LocalDate.now();

        System.out.println(today);
    }
}
```

Output:

2026-10-05

LocalDate.now()

Meaning:

LocalDate.now()

= current system date.

4. Creating a specific date

Suppose manaki:

15 October 2026

kavali.

```
LocalDate date = LocalDate.of(2026, 10, 15);
```

```
System.out.println(date);
```

Output:

2026-10-15

Syntax:

LocalDate.of(year, month, day)

Example:

```
LocalDate date = LocalDate.of(2025, 12, 25);
```

5. Important LocalDate methods

getYear()

```
LocalDate date = LocalDate.of(2026, 10, 5);
```

```
System.out.println(date.getYear());
```

Output:

2026

getMonth()

```
System.out.println(date.getMonth());
```

Output:

OCTOBER

getMonthValue()

```
System.out.println(date.getMonthValue());
```

Output:

10

Difference:

getMonth()

↓

OCTOBER

getMonthValue()

↓

10

getDayOfMonth()

```
System.out.println(date.getDayOfMonth());
```

Output:

5

getDayOfWeek()

```
System.out.println(date.getDayOfWeek());
```

Possible output:

MONDAY

6. Adding days

Suppose:

Today = October 5

5 days later?

```
LocalDate date = LocalDate.of(2026, 10, 5);
```

```
LocalDate newDate = date.plusDays(5);
```

```
System.out.println(newDate);
```

Output:

2026-10-10

Important:

```
date.plusDays(5)
```

does **not modify the original date**.

This is because Java Time API classes are **immutable**.

7. What is immutable?

Immutable means:

Once object create ayyaka, aa object value ni change cheyyalem.

Example:

```
LocalDate date = LocalDate.of(2026, 10, 5);
```

```
date.plusDays(5);
```

```
System.out.println(date);
```

Output still:

2026-10-05

Why?

Because plusDays() new LocalDate object return chestundi.

Correct:

```
LocalDate newDate = date.plusDays(5);
```

Now:

date

↓

2026-10-05

newDate

↓

2026-10-10

8. Subtracting days

```
LocalDate date = LocalDate.of(2026, 10, 5);
```

```
LocalDate previous = date.minusDays(3);
```

```
System.out.println(previous);
```

Output:

2026-10-02

Similarly:

`minusWeeks()`

`minusMonths()`

`minusYears()`

Examples:

`date.plusWeeks(2);`

`date.plusMonths(3);`

`date.plusYears(1);`

`date.minusWeeks(2);`

`date.minusMonths(3);`

`date.minusYears(1);`

9. Checking dates

Suppose:

`LocalDate date1 = LocalDate.of(2026, 10, 5);`

`LocalDate date2 = LocalDate.of(2026, 10, 10);`

isBefore()

`System.out.println(date1.isBefore(date2));`

Output:

true

Because:

October 5 < October 10

isAfter()

`System.out.println(date1.isAfter(date2));`

Output:

false

isEqual()

`System.out.println(date1.isEqual(date2));`

Output:

false

10. LocalTime

LocalTime means:

Only time, date lekunda.

Example:

18:30:45

Java:

```
import java.time.LocalTime;
```

```
LocalTime time = LocalTime.now();
```

```
System.out.println(time);
```

Output might be:

18:30:45.123456

11. Creating specific time

```
LocalTime time = LocalTime.of(18, 30);
```

```
System.out.println(time);
```

Output:

18:30

Syntax:

```
LocalTime.of(hour, minute)
```

You can also specify seconds:

```
LocalTime time = LocalTime.of(18, 30, 45);
```

Output:

18:30:45

12. LocalTime methods

```
time.getHour();
```

```
time.getMinute();
```

```
time.getSecond();
```

Example:

```
LocalTime time = LocalTime.of(18, 30, 45);
```

```
System.out.println(time.getHour());
```

```
System.out.println(time.getMinute());
```

```
System.out.println(time.getSecond());
```

Output:

```
18
```

```
30
```

```
45
```

13. Adding and subtracting time

```
LocalTime time = LocalTime.of(10, 30);
```

```
System.out.println(time.plusHours(2));
```

Output:

```
12:30
```

Other methods:

```
plusMinutes()
```

```
plusSeconds()
```

```
plusHours()
```

```
minusHours()
```

```
minusMinutes()
```

```
minusSeconds()
```

14. LocalDateTime

Now date + time together.

Date → 2026-10-05

Time → 18:30

DateTime → 2026-10-05T18:30

Java:

```
import java.time.LocalDateTime;
```

```
LocalDateTime dateTime = LocalDateTime.now();
```

```
System.out.println(dateTime);
```

Possible output:

```
2026-10-05T18:30:25.123
```

15. Creating LocalDateTime

```
LocalDateTime dateTime =
```

```
    LocalDateTime.of(2026, 10, 5, 18, 30);
```

```
System.out.println(dateTime);
```

Output:

```
2026-10-05T18:30
```

Syntax:

```
LocalDateTime.of(
```

```
    year,
```

```
    month,
```

```
    day,
```

```
    hour,
```

minute
);

16. Understand the difference

Very important interview question.

Class	Contains
LocalDate	Date only
LocalTime	Time only
LocalDateTime	Date + Time
ZonedDateTime	Date + Time + Time Zone

Example:

LocalDate

2026-10-05

LocalTime

18:30:00

LocalDateTime

2026-10-05T18:30:00

ZonedDateTime

2026-10-05T18:30+05:30[Asia/Kolkata]

17. Why LocalDate is called "Local"?

Important point.

LocalDate does **not** mean your physical location.

It means:

Date without timezone information.

For example:

```
LocalDate date = LocalDate.now();
```

It represents a calendar date, not a global instant.

Same concept for:

LocalTime

LocalDateTime

They don't contain timezone information.

PART 2 — Date Formatting

Now manam actual **practice topic: Date formatting** ki vastam.

18. What is formatting?

Suppose Java gives:

2026-10-05

But user wants:

05-10-2026

or:

05/10/2026

or:

05 October 2026

Date same.

Representation different.

This process is called:

Date formatting

19. DateTimeFormatter

Java lo formatting kosam:

DateTimeFormatter

use chestham.

Import:

```
import java.time.format.DateTimeFormatter;
```

Example:

```
LocalDate date = LocalDate.of(2026, 10, 5);
```

```
DateTimeFormatter formatter =
```

```
    DateTimeFormatter.ofPattern("dd-MM-yyyy");
```

```
String result = date.format(formatter);
```

```
System.out.println(result);
```

Output:

05-10-2026

20. Understand this code line by line

```
LocalDate date = LocalDate.of(2026, 10, 5);
```

Date create chesam.

```
DateTimeFormatter formatter =
```

```
    DateTimeFormatter.ofPattern("dd-MM-yyyy");
```

Pattern define chesam.

Meaning:

dd → day

MM → month

yyyy → year

Then:

```
String result = date.format(formatter);
```

Date ni specified format lo String ga convert chestunnam.

21. Most important formatting symbols

These symbols **case-sensitive**.

Pattern	Meaning	Example
d	day	5
dd	2-digit day	05
M	month	10
MM	2-digit month	10
MMM	short month name	Oct
MMMM	full month name	October
yy	2-digit year	26
yyyy	4-digit year	2026
E	short day name	Mon
EEEE	full day name	Monday
HH	hour (24-hour)	18
hh	hour (12-hour)	06
mm	minutes	30
ss	seconds	45
a	AM/PM	PM

22. Examples

dd-MM-yyyy

`DateTimeFormatter f =`

```
    DateTimeFormatter.ofPattern("dd-MM-yyyy");
```

Output:

05-10-2026

dd/MM/yyyy

05/10/2026

yyyy-MM-dd

2026-10-05

dd MMM yyyy

05 Oct 2026

dd MMMM yyyy

05 October 2026

EEEE, dd MMMM yyyy

Monday, 05 October 2026

23. Time formatting

```
LocalTime time = LocalTime.of(18, 30, 45);
```

```
DateTimeFormatter formatter =
```

```
    DateTimeFormatter.ofPattern("HH:mm:ss");
```

```
System.out.println(time.format(formatter));
```

Output:

18:30:45

24. 12-hour format

Suppose:

18:30

We want:

06:30 PM

Use:

```
DateTimeFormatter formatter =
```

```
    DateTimeFormatter.ofPattern("hh:mm a");
```

Example:

```
LocalTime time = LocalTime.of(18, 30);
```

```
System.out.println(time.format(formatter));
```

Output:

06:30 PM

Important:

HH → 24-hour format

hh → 12-hour format

And:

a → AM / PM

25. MM vs mm — VERY IMPORTANT

Beginners frequently make this mistake.

MM → Month

mm → Minute

Example:

dd-MM-yyyy

Correct date:

05-10-2026

But:

HH:mm

means:

18:30

Here:

HH → hour

mm → minute

Remember:

Capital M = Month

small m = minute

26. Parsing — opposite of formatting

Formatting:

Date object



String

Parsing:

String



Date object

This difference is extremely important.

27. Example of parsing

Suppose user enters:

05-10-2026

Initially it's a String.

String input = "05-10-2026";

Manages to create a LocalDate object.

DateTimeFormatter formatter =

DateTimeFormatter.ofPattern("dd-MM-yyyy");

LocalDate date =

LocalDate.parse(input, formatter);

Now:

String

"05-10-2026"



parse()



LocalDate

2026-10-05

28. Formatting vs Parsing

Remember this table:

Operation Meaning

Formatting Date → String

Parsing String → Date

Example:

```
date.format(formatter);
```

means:

Date → String

While:

```
LocalDate.parse(input, formatter);
```

means:

String → Date

29. Why parsing is useful?

Real-world application to user input usually String form to untundi.

Example:

Enter date:

05-10-2026

Scanner:

```
String input = scanner.nextLine();
```

So:

String

↓

parse

↓

LocalDate

Now manam date calculations cheyyachu:

```
date.plusDays(10);
```

```
date.isBefore(otherDate);
```

```
date.getYear();
```

30. Period

Period is mainly used to calculate difference between **dates**.

Example:

01-01-2026

to

05-10-2026

How many:

Years?

Months?

Days?

Use:

Period

Example:

LocalDate start =

```
    LocalDate.of(2026, 1, 1);
```

LocalDate end =

```
    LocalDate.of(2026, 10, 5);
```

Period period =

```
    Period.between(start, end);
```

```
System.out.println(period);
```

It represents the date-based difference.

You can get:

```
period.getYears();
```

```
period.getMonths();
```

```
period.getDays();
```

31. Duration

Duration is generally used for **time-based amounts**.

For example:

10:00

to

13:30

Difference:

3 hours 30 minutes

Use:

Duration

Example:

```
LocalTime start = LocalTime.of(10, 0);
```

```
LocalTime end = LocalTime.of(13, 30);
```

Duration duration =

```
    Duration.between(start, end);
```

Then:

```
duration.toHours();
```

```
duration.toMinutes();
```

32. Period vs Duration

Very important:

Period



Date-based difference

Duration



Time-based difference

Simple memory:

Period → Date

Duration → Time

PART 3 — Regular Expressions

Now second major topic.

33. What is Regular Expression?

Regular Expression = **Regex**

Regex ante:

Text lo manaki kavalsina pattern ni define cheyyadaniki use chese special language.

Simple ga:

"Is this text following my rule?"

ani check cheyyadaniki Regex use chestham.

34. Real-world examples

Suppose user email enter chesadu:

abc@gmail.com

Manam check cheyyali:

Idi email-la kanipistundha?

Phone:

9876543210

Check:

Exactly 10 digits unnaya?

Password:

Abc@1234

Check:

Uppercase undha?

Lowercase undha?

Number undha?

Special character undha?

All these situations lo Regex useful.

35. Java Regex classes

Mainly:

java.util.regex

package.

Important classes:

Pattern

Matcher

36. Pattern

Pattern ante:

Manam search/check cheyyali anukunna rule.

Example:

Only digits

Pattern:

[0-9]+

37. Matcher

Matcher ante:

Aa pattern ni actual text meeda apply chese object.

Think like:

Pattern



Rule

Matcher



Rule ni text meeda test chestundi

38. First simple Regex program

Suppose:

12345

contains only digits aa?

```
import java.util.regex.Pattern;
```

```
import java.util.regex.Matcher;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        String text = "12345";
```

```
        Pattern pattern =
```

```
            Pattern.compile("[0-9]+");
```

```
        Matcher matcher =
```

```
            pattern.matcher(text);
```

```
        System.out.println(matcher.matches());
```

```
    }
```

```
}
```

Output:

true

39. Understand the flow

String text = "12345";

Actual input.

Then:

Pattern pattern =

Pattern.compile("[0-9]+");

Rule create chestunnam.

Then:

Matcher matcher =

pattern.matcher(text);

Rule + text connect chestunnam.

Finally:

matcher.matches();

Check chestunnam.

40. What is [0-9]?

Meaning:

Any one digit from 0 to 9.

So:

0

1

2

3

...

9

Any one of these match avvachu.

41. What is +?

+ means:

Previous pattern **one or more times** occur avvali.

So:

[0-9]+

means:

One or more digits.

Examples:

1 ☐

12 ☐

12345 ☐

999999 ☐

But:

abc ☐

12a ☐

42. Very important Regex symbols

These are the foundation.

Regex	Meaning
-------	---------

[0-9]	one digit
-------	-----------

[a-z]	lowercase letter
-------	------------------

[A-Z]	uppercase letter
-------	------------------

[a-zA-Z]	any English letter
----------	--------------------

\d	digit
----	-------

\D	non-digit
----	-----------

\w	word character
----	----------------

\W	non-word character
----	--------------------

Regex Meaning

\s	whitespace
\S	non-whitespace
.	any character
+	one or more
*	zero or more
?	zero or one
{n}	exactly n times
{n,m}	n to m times
^	beginning
\$	end

43. [a-z]

Means:

a through z

Example:

[a-z] +

Matches:

hello

java

abc

Doesn't match:

Hello

123

hello123

because uppercase and numbers aren't included.

44. [A-Z]

Uppercase letters:

[A-Z]+

Matches:

JAVA

HELLO

ABC

45. [a-zA-Z]

Both lowercase + uppercase:

[a-zA-Z]+

Matches:

Hello

JAVA

hello

AbCd

But not:

Hello123

because digits aren't allowed.

46. \d

Instead of:

[0-9]

you can use:

\d

Meaning:

digit

So:

\d+

means:

one or more digits.

47. Important Java escaping issue

Java String lo backslash special.

Regex:

`\d+`

Java String lo usually:

`"\\d+"`

Why?

Because Java itself interprets `\`.

So:

Regex notation:

`\d+`

Java String:

`"\\d+"`

This is a very important beginner point.

48. *

* means:

zero or more times.

Example:

`[0-9]*`

Matches:

`""`

`"1"`

`"123"`

`"9999"`

Notice empty string kuda match avvachu because:

0 or more

49. + vs *

Very important.

+ → one or more

* → zero or more

Example:

`\d+`

Empty string:

X

But:

`\d*`

Empty string:

□

50. ?

? means:

zero or one time.

Example:

`colou?r`

Matches:

color

colour

Because u is optional.

51. {n}

Exactly n times.

Example:

$\backslash d\{10\}$

Means:

exactly 10 digits.

Useful for a basic 10-digit phone-number format.

9876543210 ☐

987654321 X

98765432101 X

52. $\{n,m\}$

Between n and m times.

Example:

$\backslash d\{3,5\}$

Means:

minimum 3 digits

maximum 5 digits

Examples:

123 ☐

1234 ☐

12345 ☐

12 X

123456 X

53. $^$ and $\$$

These are extremely important for validation.

$^$

means:

beginning of input

$\$$

means:

end of input

Example:

`^[0-9]+$`

Meaning:

From beginning to end, everything must be digits.

So:

12345 ☐

But:

123abc ☒

abc123 ☒

54. Why ^ and \$ matter?

Suppose regex:

`[0-9]+`

It defines digits, but depending on how you search/match, you're distinguishing the pattern from requiring the entire input.

For validation, you generally want the whole input to follow the rule:

`^[0-9]+$`

This means:

START

↓

only digits

↓

END

55. matches() vs find()

Very important Java Regex concept.

matches()

Checks whether the **entire region/input** matches the pattern.

Example:

```
String text = "12345";
```

```
Pattern pattern =
```

```
    Pattern.compile("\\d+");
```

```
Matcher matcher =
```

```
    pattern.matcher(text);
```

```
System.out.println(matcher.matches());
```

Output:

true

If:

123abc

then:

false

56. find()

find() searches for a matching **subsequence** inside the input.

Example:

```
String text = "My number is 12345";
```

```
Pattern pattern =
```

```
    Pattern.compile("\\d+");
```

```
Matcher matcher =
```

```
    pattern.matcher(text);
```

```
System.out.println(matcher.find());
```

Output:

true

Because text lo:

12345

digits unnayi.

57. matches() vs find() — remember this

matches()

↓

Entire input match avvali

find()

↓

Input lo somewhere match dorikithe enough

Example:

Text:

abc123xyz

Pattern:

\d+

matches():

false

find():

true

Because 123 exists inside.

58. Pattern.compile()

Syntax:

Pattern pattern =

```
Pattern.compile("regex");
```

It converts your regex rule into a Pattern.

Example:

```
Pattern pattern =
```

```
    Pattern.compile("\\d+");
```

59. pattern.matcher()

```
Matcher matcher =
```

```
    pattern.matcher(text);
```

It creates a Matcher for that particular text.

Then:

```
matcher.matches();
```

or:

```
matcher.find();
```

60. Character classes

Regex lo square brackets:

```
[ ]
```

character set define cheyyadaniki use chestham.

Example:

```
[abc]
```

means:

one character: a OR b OR c

It does **not** mean "abc" as a whole.

61. [abc]

Matches:

a

b

c

Not:

abc

unless you add repetition:

[abc]+

Then:

abc

aaa

bbb

cab

can match.

62. Negated character class

[^0-9]

Meaning:

Anything except digits.

The ^ inside [] has a different meaning from ^ outside.

Outside:

^

= beginning.

Inside:

[^0-9]

= NOT digits.

This distinction is important.

63. OR operator |

| means:

OR

Example:

cat|dog

Matches:

cat

or:

dog

Example:

java|python

means:

java OR python

64. Groups ()

Parentheses create a group.

Example:

(java|python)

means:

java OR python

Groups become especially useful when applying repetition or extracting pieces.

65. Email Regex — beginner version

A simple pattern:

```
^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+$
```

Java:

String regex =

```
"^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+$";
```

boolean valid =

```
Pattern.matches(regex, email);
```

This is a **basic pattern**, not a perfect RFC-level email validator.

For interview/fresher-level validation, understanding the components is more important than memorizing a giant email regex.

66. Phone number validation

For exactly 10 digits:

`^[0-9]{10}$`

Java:

`String phone = "9876543210";`

`boolean valid =`

`Pattern.matches("^[0-9]{10}$", phone);`

`System.out.println(valid);`

Output:

true

67. Username validation

Suppose requirement:

Only letters, digits and underscore

Length: 5 to 15

Regex:

`^[A-Za-z0-9_]{5,15}$`

Break it:

`^`

start

`[A-Za-z0-9_]`

allowed characters

{5,15}

length 5 to 15

\$

end

68. Password Regex

Suppose requirement:

At least:

1 uppercase

1 lowercase

1 digit

minimum 8 characters

A basic pattern using lookaheads:

`^(?=.*[A-Z])(?=.*[a-z])(?=.*\d){8,}$`

This is slightly advanced, so let's break it.

`(?=.*[A-Z])`

At least one uppercase.

`(?=.*[a-z])`

At least one lowercase.

`(?=.*\d)`

At least one digit.

`{8,}`

At least 8 characters.

69. What is .?

Regex lo:

.

means:

Any character (with some line-terminator caveats).

Example:

a.c

Can match:

abc

a1c

a-c

a c

because middle character can vary.

70. .*

Very common combination.

.*

means approximately:

Any number of characters.

Because:

. → any character

* → zero or more

So:

.*

= zero or more arbitrary characters.

71. Practical example: Student roll number

Suppose roll number format:

CSE2026

Rule:

CSE + exactly 4 digits

Regex:

`^CSE[0-9]{4}$`

Matches:

CSE2026 ☐

Doesn't match:

CSE26 ✗

ECE2026 ✗

CSE20267 ✗

72. Practical example: Date pattern

Suppose input should look like:

05-10-2026

A basic structural Regex:

```
^\d{2}-\d{2}-\d{4}$
```

This checks the **shape**:

2 digits

-

2 digits

-

4 digits

But careful:

99-99-9999

would structurally match.

Regex alone here doesn't prove that the date is a real calendar date.

For actual date validity, use:

Regex → check format

DateTimeFormatter + parsing → check actual date

This is a very useful real-world combination.

73. Regex + Date API together

Suppose user enters:

05-10-2026

Step 1:

Regex checks:

Does it look like dd-MM-yyyy?

Step 2:

LocalDate.parse() checks:

Is it actually a valid date?

Flow:

User Input

↓

Regex

↓

Correct format?

↓

DateTimeFormatter

↓

Valid LocalDate?

This is how real applications can separate **format validation** from **date parsing**.

74. Complete Date Formatting Program

```
import java.time.LocalDate;
```

```
import java.time.format.DateTimeFormatter;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        LocalDate date = LocalDate.of(2026, 10, 5);
```

```
DateTimeFormatter formatter =  
    DateTimeFormatter.ofPattern("dd MMMM yyyy");  
  
String formattedDate =  
    date.format(formatter);  
  
System.out.println(formattedDate);  
}  
}
```

Output:

05 October 2026

75. Complete Date Parsing Program

```
import java.time.LocalDate;  
import java.time.format.DateTimeFormatter;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        String input = "05-10-2026";
```

```
        DateTimeFormatter formatter =
```

```
            DateTimeFormatter.ofPattern("dd-MM-yyyy");
```

```
        LocalDate date =
```

```
            LocalDate.parse(input, formatter);
```

```
        System.out.println(date);
```

```
}  
}
```

Output:

2026-10-05

76. Complete Regex Program

```
import java.util.regex.Pattern;  
import java.util.regex.Matcher;  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        String phone = "9876543210";  
  
        String regex = "[0-9]{10}$";  
  
        Pattern pattern =  
            Pattern.compile(regex);  
  
        Matcher matcher =  
            pattern.matcher(phone);  
  
        if (matcher.matches()) {  
            System.out.println("Valid phone number");  
        } else {  
            System.out.println("Invalid phone number");  
        }  
    }  
}
```

```
}
```

Output:

Valid phone number

77. Simpler way: `Pattern.matches()`

For simple validation, you don't always need to explicitly create `Pattern` and `Matcher`.

You can write:

```
boolean result =
```

```
    Pattern.matches("^([0-9]{10})$", phone);
```

Example:

```
import java.util.regex.Pattern;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        String phone = "9876543210";
```

```
        boolean valid =
```

```
            Pattern.matches(
```

```
                "^([0-9]{10})$",
```

```
                phone
```

```
            );
```

```
        System.out.println(valid);
```

```
    }
```

```
}
```

78. When should I use `Pattern` + `Matcher`?

Use them when you're doing more advanced/repeated regex operations.

For example:

```
Pattern pattern =
```

```
    Pattern.compile("\\d+");
```

```
Matcher matcher =
```

```
    pattern.matcher(text);
```

Then you can repeatedly use:

```
matcher.find()
```

```
matcher.group()
```

```
matcher.start()
```

```
matcher.end()
```

79. group()

Suppose:

My marks are 95 and 88

Regex:

```
\d+
```

Using find():

```
String text = "My marks are 95 and 88";
```

```
Pattern pattern =
```

```
    Pattern.compile("\\d+");
```

```
Matcher matcher =
```

```
    pattern.matcher(text);
```

```
while (matcher.find()) {
```

```
    System.out.println(matcher.group());
```

```
}
```

Output:

95

88

This is useful for **extracting matching pieces** from text.

80. start() and end()

For each match, you can know where it occurs.

```
while (matcher.find()) {
```

```
    System.out.println(matcher.group());
```

```
    System.out.println(matcher.start());
```

```
    System.out.println(matcher.end());
```

```
}
```

If text:

abc123

and match:

123

then start() tells the starting index and end() tells the position immediately after the matched portion.

81. Important Regex Cheat Sheet

Character classes

[0-9] digit

[a-z] lowercase

[A-Z] uppercase

[a-zA-Z] letter

[^0-9] not digit

Predefined classes

\d	digit
\D	non-digit
\w	word character
\W	non-word
\s	whitespace
\S	non-whitespace

Quantifiers

*	0 or more
+	1 or more
?	0 or 1
{n}	exactly n
{n,m}	n to m
{n,}	n or more

Anchors

^	beginning
\$	end

Other

.	any character
	OR
()	group
[]	character class

82. Most important difference: Regex vs DateTimeFormatter

Don't confuse these two.

Regex

Used for:

Pattern validation/search

Example:

9876543210

Check:

Is it exactly 10 digits?

DateTimeFormatter

Used for:

Formatting/parsing date and time

Example:

2026-10-05

Convert to:

05-10-2026

So:

Regex

↓

Pattern

DateTimeFormatter

↓

Date/Time representation

83. Interview Questions You Should Know

Q1. What is LocalDate?

LocalDate represents a date without time or timezone.

Q2. What is LocalTime?

It represents time without date or timezone.

Q3. What is LocalDateTime?

It represents date and time without timezone.

Q4. What is DateTimeFormatter?

It is used to format date/time objects into strings and parse strings into date/time objects.

Q5. Formatting vs parsing?

Formatting:

Date → String

Parsing:

String → Date

Q6. MM vs mm?

MM → month

mm → minute

Q7. What is Regex?

Regex is a pattern language used to search, match, extract, or validate text according to specified rules.

Q8. What is Pattern?

Pattern represents a compiled regular expression.

Q9. What is Matcher?

Matcher applies a regex pattern to a character sequence and performs matching/searching operations.

Q10. matches() vs find()?

matches()

→ entire input must match

find()

→ searches for a matching portion

Q11. What does + mean?

One or more occurrences.

Q12. What does * mean?

Zero or more occurrences.

Q13. What does {10} mean?

Exactly 10 occurrences.

Q14. What does ^ mean?

Usually beginning of input.

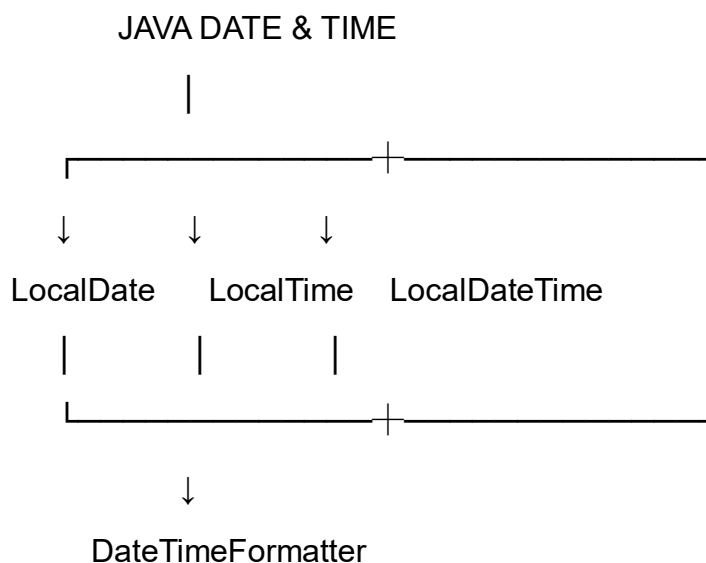
Inside a character class at the beginning, such as `[^0-9]`, it means negation.

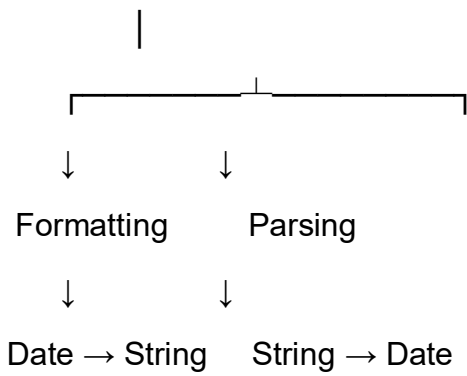
Q15. What does \$ mean?

End of input.

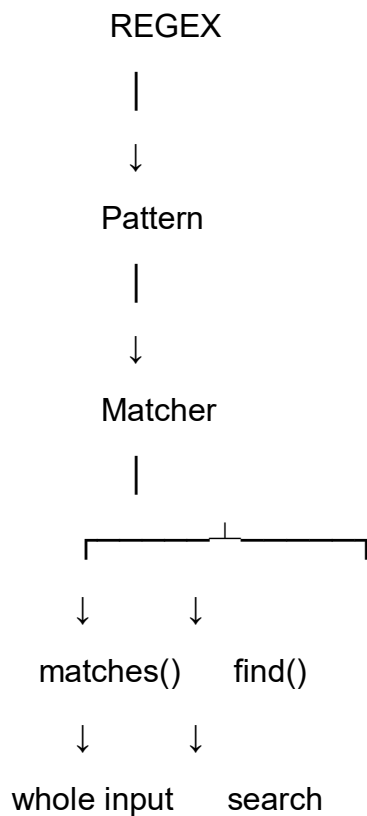
84. The complete mental picture

Now Day 39–40 ni one flow lo chuddam.





And Regex:



85. Practice Problems

Now theory complete ayindi. Ee problems solve chesthe concepts strong avuthayi.

Beginner

1. Print today's date using LocalDate.

2. Create:

25-12-2026

using LocalDate.

3. Print current time using LocalTime.

4. Create:

18:30:45

using LocalTime.

5. Add 30 days to a given date.

6. Subtract 2 months from a date.

7. Check whether one date is before another.

Date Formatting

8. Convert:

2026-10-05

to:

05-10-2026

9. Convert it to:

05 October 2026

10. Convert time:

18:30

to:

06:30 PM

11. Take a String:

15-08-2026

and convert it to LocalDate.

Regex

12. Check whether a string contains only digits.

12345 → true

123a5 → false

13. Validate exactly 10 digits.

14. Validate only uppercase letters.

15. Validate only lowercase letters.

16. Validate username:

5–15 characters

letters + digits + underscore

17. Search all numbers inside:

"My marks are 95, 87 and 92"

Expected:

95

87

92

18. Check whether a string is a basic email-like format.

86. One final thing you should remember

Don't try to memorize 100 Regex symbols immediately.

First master these:

[0-9]

[a-z]

[A-Z]

\d

+

*

?

{n}

{n,m}

^

\$

.

|

()

And Java Date/Time side:

LocalDate

LocalTime

LocalDateTime

DateTimeFormatter

format()

parse()

plusDays()

minusDays()

isBefore()

isAfter()

Period

Duration

If these are strong, **Day 39–40 is essentially covered.**

Most important 10 concepts

1. LocalDate

2. LocalTime

3. LocalDateTime

4. DateTimeFormatter

5. Formatting

6. Parsing

7. Regex

8. Pattern

9. Matcher

10. matches() vs find()

These are the concepts I would expect you to be able to **explain and code without looking up** before moving to the next Java topic.